

UNIVERSIDAD DEL VALLE DE GUATEMALA

1CC30672020262- Redes

Sección 20

Jorge Andrés Yass Coy



Proyecto #1: Propuesta

Roberto Barreda, 23354

GUATEMALA, 30 de julio de 2026

1. Caso de uso

Se propone un servidor MCP para el área de logística de una empresa de bebidas y alimentos. Cada día, el centro de distribución recibe archivos de pedidos enviados por sucursales y distribuidores, consolida esos pedidos en camiones y planifica la ruta de reparto.

Existe un problema, hoy las 3 tareas se resuelven de forma manual y en hojas de cálculo, lo que da errores de captura no detectados, camiones que salen a media carga y rutas definidas por criterio del planificador. El servidor propuesto se da para que un chatbot pueda: validar la integridad del archivo recibido, juntar los pedidos en camiones y planificar la ruta de reparto.

La ejecución local se justifica porque los archivos de pedidos, el padrón de clientes y las rutas son información interna. Con el transporte stdio, los mensajes no salen del sistema operativo del equipo del planificador: no hay medio compartido, no hay ruteo y no hay intermediarios.

2. Herramientas

Herramienta	Parámetros	Resultado
validar_recepcion	contenido (texto del archivo), crc_esperado (hex)	Estado del lote, CRC-32 calculado y lista de registros corruptos.
consolidar_carga	pedidos (lista con peso y volumen), capacidad_peso, capacidad_volumen	Asignación de pedidos por camión, porcentaje de ocupación y pedidos sin asignar.
planificar_ruta	cedis (origen), clientes (lista de destinos)	Secuencia de paradas, distancia total en kilómetros y tiempo estimado.

3. Arquitectura

El servidor se construye sobre 1 núcleo de lógica de negocio, 1 capa de protocolo JSON-RPC 2.0 implementada manualmente sin SDKs de MCP, y 2 transportes intercambiables. En modo local, los mensajes JSON-RPC viajan por stdio delimitados por salto de línea. En modo remoto, viajan como cuerpo de una petición HTTP POST al endpoint /mcp, desplegado en Vercel.

Ambos transportes invocan el mismo despachador, por lo que el chatbot utiliza el servidor remoto igual que el local.

4. Alcance de la implementación

La versión local atiende el requisito de servidor MCP propio. Segundo, la versión desplegada en Vercel atiende el requisito de servidor remoto. Tercero, el tráfico HTTPS generado por el chatbot se captura con Wireshark, descifrando TLS, para identificar los mensajes JSON-RPC de sincronización, petición y respuesta.

El servidor se da sobre un conjunto de datos de prueba embebido en el propio repositorio.